Frontend Performance Testing Framework

Design Document

Version 1.3

Status: Revised — Concurrent Load Testing Added

May 2026

Field	Value
Document Title	Frontend Performance Testing Framework — Design Document
Version	1.3
Status	Revised — Concurrent Load Testing Added
Owner	Frontend Platform / Performance Engineering
Audience	Platform engineers, SREs, frontend leads, QA, engineering leadership

Revision History

Version	Date	Author	Changes
0.1	May 2026	Platform Team	Initial framework outline (engines, architecture, dashboard, ML, gates)

Version	Date	Author	Changes
0.2	May 2026	Platform Team	Added detailed request-level timing breakdown design
0.3	May 2026	Platform Team	Added natural-language test authoring capability
1.0	May 2026	Platform Team	Consolidated design document for review
1.1	May 2026	Platform Team	Review-driven revisions: tightened Phase 1 scope, added baseline management and cost/quota sections, added semantic post-conditions for self-healing, NL safety envelope, ClickHouse cardinality controls, WPT positioning
1.2	May 2026	Platform Team	Second-review revisions: sample-based baseline stabilization, orchestration fairness (concurrency caps + LLM rate limits + weighted-fair queue), significance-aware hot-tier retention, cohort-aware bulk healing approval, explicit worker isolation, Phase 1 3-of-5 gate quorum, seasonality as schema field, LLM-off as primary privacy control, namespace throttling demoted to opt-in, post-condition auto-inference, traceparent injection as opt-in
1.3	May 2026	Platform Team	Scope expansion: concurrent-user frontend load testing and UI-tier server resource correlation. Added k6 browser as fourth engine, load-profile execution mode, UI-server telemetry ingestion (node_exporter, prom-client, nginx-exporter), load-correlation dashboard view, load-aware quality gates, load-aware ML attribution, load-specific reliability concerns, and Phase 3.5 to delivery plan

Table of Contents

1. Executive Summary

This document describes the design of a Frontend Performance Testing Framework: an end-to-end platform that lets engineers author, execute, analyze, and act on frontend performance tests. It unifies the lifecycle of synthetic performance testing — from script authoring through CI/CD quality gates — behind a single web dashboard. The platform is synthetic-first; Real User Monitoring (RUM) and CrUX field data are ingested as ground-truth correlation signals, not as primary measurement sources.

The framework targets four audiences: frontend developers who need fast feedback on performance regressions; SREs who need reliable signals and incident triage; QA who need maintainable test suites; and engineering leadership who need trend visibility and confidence that production user experience is improving over time.

Version 1.3 adds concurrent-user frontend load testing and UI-tier server resource correlation. Beyond measuring how the frontend performs for a single user, the platform now measures how it performs under realistic concurrent load — and correlates browser-side timings (LCP, INP, TTFB) with server-side resource consumption (CPU, memory, event-loop lag, GC pause) on the UI-tier servers that render or deliver the frontend. This closes a gap that single-session synthetic measurement leaves open: the difference between "the page renders fast for one user" and "the page renders fast for ten thousand concurrent users while the SSR fleet is at 80% CPU."

1.1 Key Capabilities

- Hybrid engine strategy — Playwright + Lighthouse CI as primary engines, k6 browser for concurrent load testing, WebPageTest (private instance) for deep dives, and sitespeed.io for continuous synthetic coverage.
- Three authoring modes — visual recorder, template library, and natural-language generation that turns plain English into validated Playwright scripts.
- Single-session and load-mode execution — the same canonical script can be executed in N-run stability mode for regression detection or in load mode (VU ramp profiles) for capacity verification.
- Detailed request-level timing analysis — full phase breakdown (queue, DNS, TCP, TLS, server processing, download, render) plus correlation with backend traces via W3C Trace Context propagation.
- UI-tier server telemetry — node_exporter, prom-client (event loop, heap, GC), and nginx-prometheus-exporter feed Prometheus and ClickHouse; load-run dashboard view stacks frontend timings and server resources on a shared time axis with ramp-stage annotations.
- ML-driven analysis — statistical baselines, change-point detection, anomaly feeds, and feature-attribution that explains which request phase or server resource caused a regression.
- Quality gates — budget-based, baseline-relative, statistically significant, and load-aware gates wired into GitHub, GitLab, and CI/CD systems to block bad deployments.
- Self-healing scripts — generated locators carry ranked fallbacks; runtime promotes successful fallbacks only when semantic post-conditions hold; broken scripts auto-open repair PRs.

1.2 Non-Goals

- Backend/API load testing of business logic and data tiers. The framework loads UI-tier servers because they are inseparable from frontend performance; deeper backend load testing remains out of scope and is delegated to existing backend load-test tooling.
- Mobile-native application performance (iOS/Android). Web views and mobile web are in scope; native code is not.
- Full Real User Monitoring replacement. The framework ingests RUM data for correlation but does not aim to replace dedicated RUM platforms.

2. Goals & Scope

The framework aims to be a single platform covering the full lifecycle of frontend performance work: authoring tests, executing them across realistic conditions and concurrency levels, analyzing results (including ML-based anomaly and regression detection), reporting trends, and enforcing quality gates in CI/CD.

2.1 Primary Objectives

- Lower the cost of authoring and maintaining performance tests through visual recording, templates, and natural-language generation.
- Produce reliable, reproducible, multi-environment measurements covering Core Web Vitals and custom KPIs.
- Measure frontend performance under realistic concurrent user load, not only single-session timing.
- Correlate browser-side performance metrics with UI-tier server resource consumption so regressions can be attributed to client-side, server-side, or capacity causes.
- Detect regressions automatically and block bad deployments via quality gates.
- Provide trend analysis, root-cause hints, and executive-friendly reporting.

2.2 In Scope

- Synthetic frontend performance measurement (single-session and load mode).
- UI-tier server resource and application metrics (CPU, memory, event-loop lag, GC, SSR render time, static-delivery throughput) — strictly to enable correlation with frontend performance.
- CI/CD integration including gating, scheduled runs, and PR-triggered execution.
- Dashboards for run-level analysis, comparison, trends, anomalies, root-cause attribution, and load-correlation views.
- Ingestion of RUM and CrUX field data for ground-truth correlation.

2.3 Non-Goals

- Backend/API load testing of business logic, data tiers, or services beyond the UI-tier. The framework loads UI servers because they are part of the frontend critical path; deeper backend capacity testing belongs to existing backend load-test tooling.
- Mobile-native (iOS/Android) application performance.
- Full RUM replacement. The framework consumes RUM as a correlation signal but does not replace dedicated RUM platforms.
- Functional or visual regression testing beyond the minimum needed to verify that performance measurements were taken against the intended journey (semantic post-conditions for self-healing fall within scope).

3. Engine Selection

WebPageTest (WPT) is a strong baseline — private-instance support, rich waterfall data, filmstrips, and broad trust. But a single tool cannot serve every measurement need at acceptable cost. The framework therefore takes a hybrid approach, with a clear role for each engine and a unified TestRunner abstraction underneath.

3.1 Selected Engines and Their Roles

Engine	Role	Execution Mode	Notes
Playwright + Lighthouse CI	Primary engine for single-session synthetic runs in CI/CD	N-run stability	Hot path. Used by default for PR gating, scheduled checks, and trend baselines. Modern CDP access, fast, deterministic with N-run medians.
k6 browser	Concurrent-user load testing of the frontend	Load profile (VU ramp)	Added in v1.3. Real Chromium under k6's VU scheduler; native Prometheus output that aligns with server-side telemetry on the same metric backend.
WebPageTest (private)	Deep-dive forensic analysis	Opt-in, rate-limited	Used for filmstrip/waterfall investigation, multi-location testing, and accurate connectivity throttling. Not on the every-PR path.
sitespeed.io	Continuous synthetic coverage	Scheduled	Long-horizon trend data; integrates with Grafana and complements Lighthouse runs.

CrUX and RUM data are ingested as ground-truth correlation signals against which synthetic results are validated, not as their own engines.

3.2 Why k6 Browser for Load Mode

Several tools can drive concurrent browser sessions — Playwright can be parallelized via cluster orchestration, Selenium Grid handles browser farms, commercial services like BrowserStack run cloud browsers. k6 browser is chosen as the framework's load-mode engine for three reasons:

- Native VU scheduling. k6 has first-class virtual-user semantics (ramping-vus, constant-vus, ramping-arrival-rate) that produce reproducible load profiles. Playwright clusters can replicate this but require custom orchestration.
- Native Prometheus output. k6 emits its metrics directly to Prometheus via remote write. The same Prometheus instance scrapes node_exporter and prom-client on UI servers, so frontend and server metrics share one backend and one time axis without an intermediate ingestion layer.

- Convergent script format. k6 browser uses a Playwright-compatible API. Scripts written for single-session Playwright runs can be adapted to k6 browser load runs with limited changes, preserving the framework's principle that one canonical script serves multiple execution modes.

Playwright remains available as a load engine for cases where the team already has Playwright tooling investments and wants to reuse it; the TestRunner abstraction supports both.

3.3 Engine Abstraction

All engines are wrapped behind a thin internal TestRunner interface. A run specification carries the script (canonical format), the engine selection, the execution mode (stability / load / deep-dive / scheduled), and the environment context. The orchestrator dispatches to the appropriate engine; results are normalized into the same schema regardless of source.

This keeps the framework future-proof. New engines — a future Puppeteer integration, a commercial deep-dive engine, or a different load engine — can be added by implementing the interface without changing the dashboard, storage, or gate layers.

3.4 WPT Operational Positioning

WPT is positioned as an opt-in, rate-limited deep-dive engine, not as something every team gets by default. Private WPT carries ongoing operational cost — agent health, browser version drift across agents, connectivity-profile maintenance, geographic distribution. Treating it as an on-demand investigation tool rather than a per-PR hot-path engine keeps that cost manageable.

4. System Architecture

The platform is a multi-service system. Components are deployed on Kubernetes, communicate over a message bus, and share Postgres for metadata, ClickHouse for high-volume time-series and per-request data, and an object store for artifacts.

4.1 Component Overview

Component	Responsibility
API Gateway	Public REST/GraphQL surface; OIDC; RBAC; CLI/SDK entry point
Authoring Service	Script CRUD, versioning, NL generation pipeline, recorder backend, template management
Orchestrator	Run scheduling, worker dispatch, fairness queue, concurrency caps, cost estimation, quota enforcement
Workers (per engine)	Stateless containers that execute runs and emit metrics + artifacts. Includes Playwright/Lighthouse, k6 browser, WPT bridge, sitespeed.io
Telemetry Ingestion	Scrapes Prometheus targets (k6, node_exporter, prom-client, nginx-exporter) and ingests Server-Timing / trace context
Analytics Service	Baseline computation, statistical tests, change-point detection, root-cause attribution
Gate Service	Evaluates quality gates against run results and writes back to CI
Dashboard (Next.js)	Single web UI: author, run, analyze, insights, report, gates, admin
Notification Service	Slack, email, webhook delivery of digests and alerts

4.2 Data Plane

- Postgres — metadata: projects, users, scripts, run records, gate configs, baselines, audit log.
- ClickHouse — metrics and per-request timing rows; load-run server resource series; sized for high-cardinality time-series with explicit retention tiers (see §11.6).
- Prometheus — short-term metrics store for live dashboards, especially during in-flight load runs.
- Object store (S3-compatible) — HARs, traces, filmstrips, videos, screenshots, k6 raw output bundles.
- Kafka (preferred) or Redis Streams — event bus for run lifecycle, metric ingestion, and gate evaluation.

4.3 Worker Isolation Model

Workers are pinned to dedicated node pools by engine type. Playwright/Lighthouse workers, k6 browser load generators, and WPT bridge workers do not co-schedule on the same nodes. This is required for measurement integrity:

- Single-session synthetic workers must run in a low-noise environment; co-located load generators would distort measurements.

- k6 browser load generators are CPU-bound under high VU counts; co-locating them with other workers would cause cross-contamination of resource metrics and increase variance.
- WPT agents are long-lived and have their own browser/network profile state; mixing them with ephemeral workers complicates lifecycle management.

Anti-affinity rules and dedicated node pools (taints/tolerations) enforce the separation. Pod-level resource requests are set to match the node's allocatable capacity so the scheduler cannot pack additional pods onto a synthetic worker node.

4.4 Orchestration Fairness

Multiple concurrency controls prevent thundering-herd starvation when many teams trigger runs simultaneously (e.g., during a pre-release freeze):

- Per-project concurrency caps — hard ceiling on simultaneous runs from any one project, set on creation and adjustable by admins.
- Weighted-fair queue across projects — when the worker pool is saturated, runs from projects below their fair share preempt new arrivals from projects above it.
- LLM rate limits — the NL authoring path goes through a shared LLM endpoint; per-project token-bucket rate limits prevent one team's bulk-generation from starving others.
- Engine-specific caps — separate quotas for k6 browser load runs (which consume far more resources per run than single-session runs), so a large load test does not displace dozens of PR-gating runs.

4.5 Telemetry Ingestion Architecture

The framework collects telemetry from three distinct sources, merged into one queryable backend:

- Browser-side (per-run) — Playwright/k6 browser/WPT emit page-level metrics (LCP, INP, CLS, FCP, TTFB, custom marks) and per-request timing rows (queue, DNS, TCP, TLS, server processing, download). These are pushed to ClickHouse via the workers.
- UI-tier server-side (continuous) — Prometheus scrapes node_exporter (OS), prom-client (Node.js: event loop, heap, GC), and nginx-prometheus-exporter (static delivery) on every UI server. A configured target list per project tells Prometheus which servers belong to which test environment.
- Load-generator-side — the load generators running k6 browser are themselves monitored via node_exporter, so generator saturation can be excluded as a confound when correlating load-run results.

All three streams share the same Prometheus instance during a run; ClickHouse pulls the load-run window's data via a Prometheus-to-ClickHouse exporter (or remote-write) for long-term storage. Time alignment relies on NTP-synced clocks across all hosts; the orchestrator verifies clock skew at run start and aborts if any host exceeds 100 ms drift.

5. Dashboard

A single web dashboard is the primary interface for all roles. The dashboard exposes the framework's seven workflows: authoring, running, analyzing, surfacing insights, reporting, managing gates, and administration.

5.1 Authoring

- Three authoring entry points — Describe (natural language), Record (visual recorder), Template (curated library).
- Monaco-based script editor with engine-aware autocomplete and the canonical script format on save.
- Inline preview of the test plan and any clarifying questions raised by the NL pipeline.
- Versioning via Git; every script change is a reviewable commit.

5.2 Running

- Runs list with filters by project, environment, engine, mode (stability / load), and time range.
- Live view for in-flight runs — shows ramp progress, current VU count, live frontend metrics, and live server resource panels.
- Schedules and matrices — define cron-based runs and parameter sweeps (device × network × geo).
- Manual run launcher — pick a script, environment, mode, and load profile (if applicable); see cost estimate before submitting.

5.3 Analyzing

- Per-run view — metrics summary, request waterfall, filmstrip, trace download, server resource panels (load runs only).
- Compare view — side-by-side of two runs, highlighting per-metric and per-phase deltas.
- Trend view — time-series of any metric across runs with optional baseline overlay.
- Waterfall view — per-request phase breakdown with Server-Timing and trace links.
- Filmstrip view — frame-by-frame visual progression with paint events annotated.
- Load-correlation view (added in v1.3, see §12) — stacked time-series panels aligning frontend timings with UI-server resource consumption.

5.4 Insights

- Anomalies feed — runs flagged by the analytics service with severity and suspected cause.
- Root cause — per-anomaly explanation citing the phase, request, server resource, or code path most associated with the regression.
- Forecasts — projected trend over the next N days based on historical data.
- Third-party panel — per-vendor request volume, weight, and timing impact.

5.5 Reporting

- Executive dashboards — rolling 7/30/90-day Core Web Vitals, pass-rate, regression count.
- Per-team scorecards — ownership-based views mapped via CODEOWNERS or a config file.
- Exportable reports — PDF, scheduled email digests, Slack weekly summaries.

5.6 Quality Gates

- Budget editor — define thresholds per page, route, or metric (absolute or relative to baseline).
- Load-gate editor (added in v1.3) — define thresholds parameterized by VU count: "p95 LCP under 2.5 s at up to 100 concurrent VUs."
- Gate policy editor — which gates block merges, which only warn, which page on-call.
- CI integration status — pending/passed/failed checks per PR with deep links into the dashboard.
- Override workflow — explicit, audited override path with required justification (gates that cannot be bypassed get disabled by teams).

5.7 Information Architecture

Nav Section	Sub-sections	Primary Users
Author	Describe / Record / Template / Library	Devs, PMs, QA
Run	Runs list / Live view / Schedules / Matrices	Devs, QA, SREs
Analyze	Per-run / Compare / Trends / Waterfall / Filmstrip / Load-correlation	Devs, SREs
Insights	Anomalies / Root cause / Forecasts / Third parties	SREs, leads
Report	Executive / Per-team / Scheduled exports	Leadership, leads
Gates	Budgets / Load gates / Policies / CI status / Overrides	Devs, SREs, leads
Admin	Projects / Users / Roles / Integrations / Audit	Admins

6. Request-Level Timing Breakdown

A core capability of the framework is detailed, per-request timing analysis. For every request a page makes — HTML, JS, CSS, images, XHR/fetch, WebSocket upgrades — the framework captures the full phase breakdown the browser exposes through the Resource Timing API, Navigation Timing API, and Chrome DevTools Protocol (CDP), plus server-side correlation via Server-Timing headers and W3C Trace Context propagation.

> **Why This Matters**
>
> Page-level metrics like LCP tell you something is slow. Phase-level timings tell you where the time went — queueing, DNS, TCP, TLS, server processing, content download, or rendering. Without this granularity, root-cause analysis stalls at the symptom.

6.1 Per-Request Phases Captured

Phase	Definition	Source
Queueing	Time the browser held the request before dispatching (connection limits, priority)	CDP Network.timing
Stalled / blocked	Waiting for socket, proxy negotiation, or higher-priority request	CDP Network.timing
DNS lookup	domainLookupStart → domainLookupEnd	Resource Timing
TCP connect	connectStart → connectEnd (excluding TLS)	Resource Timing
TLS handshake	secureConnectionStart → connectEnd	Resource Timing
Request sent	requestStart → first byte sent	CDP / Resource Timing
Server processing	requestStart → responseStart (DevTools 'waiting')	CDP / Resource Timing
Content download	responseStart → responseEnd	Resource Timing
Service worker	workerStart → fetchStart when SW intercepts	Resource Timing
Redirects	redirectStart → redirectEnd including chain length	Resource Timing
Cache outcome	Derived from transferSize / encodedBodySize / decodedBodySize	Resource Timing
Protocol / reuse	HTTP version, ALPN, nextHopProtocol, connection reuse flag	CDP / Resource Timing

6.2 Per-Page Phases Captured

- Unload of previous page, redirect chain, app cache, DNS/TCP/TLS for the document.
- Server response (document TTFB).

- DOM processing — responseEnd → domInteractive → domContentLoadedEventStart/End → domComplete.
- Rendering — parse, style, layout, paint, composite (extracted from the trace, not Navigation Timing).
- Script execution time — JS compile and execute, broken down by script URL.
- Long tasks (>50 ms) blocking the main thread, with attributed scripts.
- Load event — loadEventStart → loadEventEnd.

6.3 Server-Side Correlation

Browser-side timings stop at responseStart — they cannot tell you what the server did during that time. Two mechanisms close the loop:

- Server-Timing headers — backend phases like db;dur=42, cache;dur=3, render;dur=88 are surfaced in the waterfall. The browser exposes Server-Timing to JS for same-origin (and cross-origin with TAO). The collector reads these from CDP response headers and stores each metric as a child timing of the request's server-processing phase.
- W3C Trace Context propagation — the Playwright runner can inject traceparent on the navigation request (opt-in per project). The script may also opt in for XHR/fetch via a route handler. Each network row in the waterfall then links to the backend distributed trace in Jaeger/Tempo/Datadog.

6.4 Collection Pipeline

Layer A — Browser APIs via Playwright/CDP

- Subscribe to CDP Network.* events on every page: requestWillBeSent, requestWillBeSentExtraInfo, responseReceived, responseReceivedExtraInfo, loadingFinished, loadingFailed. These give the authoritative timing object with the same fields DevTools uses.
- Subscribe to Page.* and Performance.* for navigation milestones and PerformanceObserver entries (paint, largest-contentful-paint, layout-shift, longtask, event).
- Inject a small in-page script that snapshots performance.getEntriesByType('resource'|'navigation'|'paint'|'mark'|'measure') at end-of-test as a cross-check and to capture entries CDP may have missed for cross-origin resources without TAO.

Layer B — Chrome trace (DevTools Performance)

- Start a trace via Tracing.start with categories devtools.timeline, disabled-by-default-devtools.timeline, loading, v8.execute, blink.user_timing. This is the source for rendering breakdown (Recalculate Style, Layout, Paint, Composite Layers), JS sampling, long tasks, and the filmstrip.
- Post-process with a trace parser to attribute time per script URL and per render phase.

Layer C — Network-layer ground truth

- Run the browser behind a controlled proxy (e.g., mitmproxy sidecar or a custom Go proxy) when high-fidelity TLS/DNS timing or HTTP/2/3 frame-level inspection is needed.

- For WPT runs, ingest the WPT HAR and page-data JSON directly — WPT already produces this layered view; the framework normalizes it into the same schema.

6.5 Storage Model

Per-request rows are stored in ClickHouse with the schema below. Page-level metrics live in a separate fact table joined by run_id.

```
request_timings (
  run_id           UUID,
  page_url         String,
  request_url      String,
  initiator_type   LowCardinality(String),  -- document, script, img, xhr, fetch, ...
  protocol         LowCardinality(String),  -- h1, h2, h3, ...
  connection_reused UInt8,
  cache_outcome    LowCardinality(String),  -- miss, hit, validated, sw
  start_time_ms    Float64,
  -- phase durations in milliseconds
  queue_ms         Float64,
  stalled_ms       Float64,
  dns_ms           Float64,
  tcp_ms           Float64,
  tls_ms           Float64,
  request_sent_ms  Float64,
  server_ms        Float64,
  download_ms      Float64,
  total_ms         Float64,
  -- sizes
  transfer_size    UInt32,
  encoded_size     UInt32,
  decoded_size     UInt32,
  -- server-side correlation
  server_timing    String,   -- raw header
  traceparent      String,
  -- attribution
  is_render_blocking UInt8,
  is_critical_path   UInt8,
  -- envelope
  ingested_at      DateTime
) ENGINE = MergeTree
ORDER BY (run_id, page_url, start_time_ms)
PARTITION BY toYYYYMM(ingested_at)
```

6.6 Cardinality Controls

Per-request data is the largest cardinality source in the system. A naive design captures every request from every run, which on a busy organization can reach tens of millions of rows per day. The framework applies several controls:

- Top-N request persistence — by default the slowest, largest, and render-blocking requests are kept individually; the rest are aggregated by initiator type and origin into a single rolled-up row per (run, origin, type).
- Critical-path preservation — requests on the LCP critical path are always kept individually, regardless of N.
- Third-party aggregation — requests to known third-party origins (ad networks, analytics, CDNs) are aggregated by vendor entity using a maintained domain-to-entity map.

- Sampling for continuous synthetic — scheduled runs (sitespeed.io continuous monitoring) apply per-run sampling: full detail on a periodic subset, aggregated detail on the rest.
- Significance-aware hot-tier retention — runs that triggered a gate failure or anomaly stay in the hot tier longer than uneventful runs, so high-value forensic data is always queryable without warm-tier rehydration.

6.7 Storage Tiers

Tier	Backend	Retention	Contents
Hot	ClickHouse (SSD)	14 days (90 days for runs with gate failure or anomaly)	Full per-request detail; queried by dashboard and gates
Warm	ClickHouse (HDD or compressed)	180 days	Aggregated per-run summaries; full detail for significant runs
Cold	S3 (Parquet)	2 years	Compressed Parquet; restored on demand for historical investigation

6.8 Waterfall Dashboard View

- Filterable by initiator type, render-blocking flag, third-party vendor, and phase contribution.
- Each row expands to show per-phase durations as a stacked bar with Server-Timing children inline.
- Trace context links open the backend distributed trace in the configured APM tool.
- Hover surfaces protocol, connection reuse, cache outcome, and transfer/encoded/decoded sizes.

6.9 Root-Cause Attribution

The ML layer (Section 9) uses per-phase data as features for regression attribution. When a page metric (e.g., LCP) regresses, the attributor identifies which request and which phase contributed most to the delta. The output is a sentence like "LCP regressed 320 ms; 78% attributed to server processing on the hero image request (was 90 ms, now 340 ms)."

6.10 Known Limitations

- Throttling fidelity — browser-level CDP throttling distorts queueing measurements. For accurate queue and stall numbers, throttle at the proxy or container network namespace (tc netem), not via CDP.
- TAO header gaps — without Timing-Allow-Origin, cross-origin resources expose only startTime and responseEnd. The framework detects this, surfaces a warning in the UI ("18 third-party resources have no detailed timing — ask the vendor to set TAO"), and falls back to proxy-derived timings where possible.

6.11 Success Metrics

Metric	Target
Time-to-root-cause on regression (gate-fail to phase-named comment)	Under 5 minutes for cases where data exists
Coverage — % of LCP-contributing requests with full phase timings	Increase over time; track as KPI
Attribution accuracy — ML root-cause agrees with human RCA	80%+ agreement on sampled incidents

7. Natural-Language Test Authoring

Authoring is the largest source of friction in performance testing — scripts break when selectors change, new team members cannot write them quickly, and PMs or SREs who know what to measure cannot express it in code. The framework offers a third authoring mode alongside Record and Template: natural-language generation that converts plain English into a validated Playwright script with appropriate performance assertions wired in.

> **Example Input**
> "Log in as a returning shopper, search for 'wireless headphones', add the top result to cart, and measure LCP on the product page." The system produces a working, parameterized script in under 15 seconds for a familiar page.

7.1 Positioning

NL generation is a third authoring mode, not a replacement for Record or Template. All three modes converge on the same canonical script format, so users can switch between modes mid-edit. Record is best for "show me," Template for "pick one," and Describe for "explain it."

7.2 User Experience

1. User types intent in a prompt box. Optional inline fields appear for target URL/environment, device/network profile, and success metrics.
2. User clicks Generate. The system produces a draft script, a human-readable step list, and clarifying questions for anything ambiguous.
3. User reviews and edits — either in plain English (regenerate) or directly in the script editor (manual edit).
4. User runs the draft once to verify; the system stores both the NL prompt and the generated script as a versioned pair.

7.3 Six-Stage Generation Pipeline

Stage	Purpose	Output
1. Intent parse	Convert NL into a structured intent tree: actions, parameters, assertions	JSON intent
2. Page reconnaissance	Crawl the target page with a headless browser, extract stripped accessibility tree	Sanitized a11y tree
3. Locator synthesis	For each action, generate primary locator + ranked fallbacks grounded in the a11y tree	Locator candidates
4. Script assembly	Compose the canonical script with locators, assertions, and metric capture	Draft script
5. Static validation	Lint, type-check, dry-run with a headless browser against the same target	Validation report

Stage	Purpose	Output
6. Confidence + clarify	Score each step's confidence; surface clarifying questions for low-confidence steps	User-facing diff

Crucially, Stage 2 grounds the model in the stripped accessibility tree, not the raw DOM. The a11y tree is far smaller, more stable across releases, and aligns with how users actually identify UI elements (by role and name). This is the single most important design choice for keeping generated scripts robust.

7.4 Canonical Script Format

The framework stores every script — regardless of authoring mode — in a canonical JSON-on-disk format that compiles to executable Playwright code. The canonical format carries: step metadata (the NL intent for each step), the primary locator, ranked fallback locators, semantic post-conditions, and per-step performance assertions.

```
{
  "id": "checkout_flow",
  "source_prompt": "Log in as a returning shopper...",
  "steps": [
    {
      "intent": "click sign-in button",
      "locators": {
        "primary": { "role": "button", "name": "Sign in" },
        "fallbacks": [
          { "text": "Log in" },
          { "testid": "header-signin" }
        ]
      },
      "post_conditions": {
        "url_pattern": "/login",
        "expect_xhr": { "method": "GET", "url_pattern": "/api/session" }
      }
    }
    // ... more steps
  ],
  "metrics": {
    "page": "product_detail",
    "assertions": [{ "metric": "lcp", "p95_max_ms": 2500 }]
  }
}
```

7.5 Self-Healing Scripts

Generated scripts will outlive the pages they were written against. Healing is built into the runtime, not just authoring — but with strict semantic guardrails. A locator that resolves is not the same as a step that did the right thing, and silent semantic drift is the single largest risk in any self-healing system.

> **Critical Safety Rule**
>
> A fallback locator is never accepted on the basis of resolution alone. Every step carries semantic post-conditions — expected URL transition, expected DOM landmark, expected outgoing API call, or expected accessibility role — that must also hold before the step is

> considered successful. Without this rule, a fallback can click the wrong UI variant (e.g., "Save draft" instead of "Save and publish"), produce green metrics on the wrong journey, and silently corrupt baselines.

Locator Strategy

- Each locator carries ranked fallbacks from Stage 3.
- On failure, the runner tries fallbacks in priority order.
- Each step carries explicit post-conditions captured at authoring time: expected URL pattern after navigation, expected DOM landmark or accessibility role, expected outgoing XHR/fetch (method + URL pattern), or a user-defined assertion.
- A step is only considered successful — and a fallback only eligible for promotion — when both the locator resolved AND the post-conditions held.

Post-Condition Auto-Inference

Manually authoring post-conditions for every step is friction; the system auto-infers them at Stage 4 by observing the page during recon: which URL did clicking the element lead to, which API call fired, what landmark became visible. The user reviews and edits the inferred post-conditions in the script editor, but defaults are derived automatically.

Promotion Workflow

- If a fallback succeeds and post-conditions pass, the run is marked Healed; the dashboard shows a diff: "Step 3 used fallback locator getByText('Add to bag') instead of primary getByRole('button', {name: 'Add to cart'}). Post-conditions verified (URL → /cart, expected POST /api/cart). Promote fallback?"
- Promotion is never automatic. One click promotes the fallback to primary and commits the change to Git with a reviewable diff.
- If a fallback resolves but post-conditions fail, the step is marked Drifted — not Healed — and the run fails with a high-confidence semantic-drift alert. The script is flagged Needs Repair.
- If all locators fail outright (no fallback resolves), the script is flagged Needs Repair. A background job re-runs the generation pipeline against the current page and opens a PR with the proposed fix, using the original NL prompt as the source of truth.

Cohort-Aware Bulk Healing Approval

When a deploy invalidates a common locator across many scripts (e.g., a global header redesign changes the sign-in button), dozens of healing PRs can land simultaneously. The dashboard groups these into cohorts by the underlying selector change, so a reviewer approves the cohort once rather than approving each PR individually. Cohort approval still produces individual commits per script for audit trail.

The original NL prompt stored at authoring time is what makes auto-repair possible — the system can always re-derive the script from intent when the page changes. Semantic post-conditions are what make healing safe rather than a quiet source of corrupted measurements.

7.6 Confidence Scoring and Clarification

- Each generated step has a confidence score from Stage 6 (0–1) based on locator uniqueness, intent ambiguity, and validation success.
- Steps with confidence below a configurable threshold (default 0.7) trigger a clarifying question to the user before the script is committed.
- Example: "Two elements match 'Add to cart' — the main product button and a quick-add button on the related items shelf. Which did you mean?"
- The user's answer is fed back into Stage 3 and the script regenerated with the disambiguating locator pinned.

7.7 Versioning and Audit

- Every generated script is committed to Git as a pair: the canonical script JSON and the source NL prompt as a sibling .prompt.md file.
- Subsequent edits — whether NL regeneration or manual — produce diffs on both files.
- The audit log records prompt, generator model version, generation time, validation outcome, and human review status for every script revision.

7.8 Runtime Automation Policy Envelope

NL generation is a powerful capability and a meaningful expansion of attack surface. The runtime enforces a policy envelope on every generated script regardless of how it was authored:

- Domain allowlist — scripts can only navigate to domains explicitly allowed for the project. Cross-domain navigation requires admin approval.
- Environment restriction — production targets are opt-in per project; staging is the default. Running against production requires explicit project-level enablement and prints an audit-log line.
- Destructive-action denylist — patterns matching destructive intents (DELETE method to non-cart endpoints, click on text matching "delete account", forms targeting admin domains) are blocked at runtime and require a manual override flag.
- Navigation boundaries — a script that attempts navigation outside the allowlist is aborted with a logged failure, not silently allowed.
- Rate limits — per-script and per-project request budgets prevent a runaway script from hammering a target.
- Side-effect classification — every step is tagged read / mutate / destructive at generation time; mutate and destructive steps require explicit acknowledgement in the dashboard before the script can be saved.

This envelope is enforced by the runner, not by the generator. A jailbroken LLM cannot bypass it; the worker refuses to execute steps outside the envelope.

7.9 Security

- The LLM never sees real credentials. Credentials are referenced by name (e.g., shopper_a) and resolved at runtime from the secrets store.
- The generator is sandboxed: it browses the target site from a workers-only network, and its outputs are reviewed (PR diff) before merging into the main branch.

- LLM-off mode — projects can disable NL authoring entirely. This is the primary privacy control for teams that cannot route page content through an external model.
- Self-hosted model option — for teams with privacy requirements but who want NL authoring, the generator supports self-hosted open models in addition to commercial APIs.

8. Data Model

The framework's data model is split across Postgres (metadata, low-cardinality) and ClickHouse (metrics and per-request data, high-cardinality). Object storage holds artifacts. The model is designed so that any run can be queried by run_id and joined across all stores.

8.1 Postgres Schema (metadata)

- projects — id, name, owner_team, environment configs, quota, allowlists.
- scripts — id, project_id, canonical JSON, source_prompt, version (git ref), authoring_mode.
- runs — id, script_id, mode (stability / load / deep / scheduled), engine, environment, started_at, finished_at, status, cost_actual.
- baselines — id, scope (project, script, metric), value, computed_at, valid_from, valid_to, sample_size, confidence, seasonality_profile.
- gates — id, project_id, definition (JSON), policy (block / warn / page), override_audit.
- audit_log — actor, action, target, timestamp, payload.

8.2 ClickHouse Tables (metrics and per-request)

- page_metrics — run_id, page_url, lcp, fcp, inp, cls, ttfb, custom metrics; partitioned by month.
- request_timings — per-request phase rows (see §6.5).
- server_resources — added in v1.3: time-series of UI-server CPU, memory, event-loop lag, GC, SSR render duration, keyed by run_id, host, and timestamp.
- trace_correlations — request_id ↔ traceparent ↔ backend trace_id for join into APM.
- anomalies — analytics service output; keyed by run_id and metric.
- All tables use MergeTree-family engines with monthly partitioning and ORDER BY clauses optimized for typical query patterns.

8.3 Object Store Layout

- artifacts/<run_id>/playwright_trace.zip — full Playwright trace.
- artifacts/<run_id>/lighthouse.json — Lighthouse report.
- artifacts/<run_id>/har.json — HTTP Archive for the run.
- artifacts/<run_id>/filmstrip/*.jpg — frame captures.
- artifacts/<run_id>/k6_summary.json — k6 raw output (load runs).
- artifacts/<run_id>/server_metrics_snapshot.parquet — snapshot of relevant Prometheus series (load runs).

8.4 Cardinality Strategy

The dominant cardinality drivers are request_timings and server_resources. Controls:

- Top-N persistence and third-party aggregation for request_timings (§6.6).
- Per-run sampling caps for continuous scheduled runs.

- Significance-aware hot-tier retention — runs that failed a gate or triggered an anomaly stay in the hot tier 90 days instead of 14.
- Server resource metrics use Prometheus's native downsampling (recording rules) for long-term retention, with full-resolution data kept only for the load-run window in ClickHouse.

9. ML Analysis

The analytics service detects regressions, attributes root cause, and forecasts trends. It is intentionally simple to start with — statistical baselines and change-point detection — and grows into feature attribution and forecasting only once enough historical data exists to make those models meaningful and operable.

9.1 Statistical Baselines

- Per-metric baselines computed from a rolling sample of recent stable runs (excluding runs marked invalid by deploys, incidents, or manual overrides).
- Baselines store p50, p75, p95, p99, mean, and standard deviation, plus a confidence score derived from sample size and variance.
- Day-of-week and time-of-day seasonality stored as schema fields on the baseline row; a baseline carries one set of values per seasonality bucket.
- Sample-based stabilization (added in v1.2) — new baselines are formed once a target sample size of stable runs is reached in the relevant seasonality bucket, rather than after a fixed 7-day window. This avoids the trap of perpetual "stabilizing" state in high-velocity continuous deployment.

9.2 Change-Point Detection

- CUSUM and EWMA detectors run on every page metric stream.
- Detections are scored by magnitude and persistence; ephemeral spikes are suppressed unless they cross a hard floor.
- A detected change point opens an anomaly in the dashboard with the suspected change time, the metric, and the suspected scope (single page, entire project, or global).

9.3 Root-Cause Attribution

- Phase-level heuristic attribution — for each anomaly on a page metric, the attributor compares per-phase distributions before and after the change point and identifies the phase(s) with the largest distributional shift.
- Request-level attribution — within the implicated phase, the attributor identifies the specific request(s) whose timing shifted most.
- Server-side attribution (added in v1.3) — for load runs, the attributor also evaluates UI-server resource series in the same window. A regression that coincides with an event-loop-lag spike or a CPU saturation event is attributed to capacity rather than to a client-side change.
- SHAP-based feature attribution is added in Phase 4 when enough historical data exists to train meaningful models. Until then, the heuristic attributor is the production path.

9.4 Forecasting

- Forecasts are added in Phase 4 only. Prophet-style models project Core Web Vitals trends over the next N days based on historical data and known seasonality.
- Forecasts are displayed alongside baselines on trend charts; they do not directly drive gates.

9.5 Explainability

- Every anomaly and attribution carries a human-readable explanation.
- Example: "LCP regressed 320 ms on /product. 78% of the delta is attributed to server processing on the hero image request; the corresponding backend trace shows a database query that previously hit cache now goes to the primary."
- Users can mark attributions as correct or incorrect; mismatches feed back into the attributor's heuristic tuning and into the future SHAP training set.

10. Quality Gates

Quality gates wire performance signals into the deployment pipeline. The framework supports four gate types: threshold, baseline-relative, statistical, and load-aware. Each can be configured to block merges, warn only, or page on-call.

10.1 Gate Types

Type	Definition	Use Case
Threshold	Absolute floor or ceiling on a metric (e.g., p95 LCP ≤ 2500 ms)	Hard SLAs, contractual obligations
Baseline-relative	No metric may regress more than X% vs. main branch baseline	Per-PR regression prevention
Statistical	Regression must be significant at $p < 0.05$ over N runs	Avoids flagging noise as failures
Composite	Combines multiple gates with AND/OR (e.g., perf score ≥ 90 AND LCP ≤ 2.5 s)	Lighthouse score plus a specific metric
Load-aware (v1.3)	Threshold parameterized by VU count (e.g., p95 LCP ≤ 2.5 s at up to 100 VUs)	Capacity verification before deploy

10.2 Phase 1 Quorum Requirement

In Phase 1, the system uses a 3-of-5 quorum: a regression must appear in at least three of the last five runs to fail a gate. This is the primary defense against synthetic flakiness while the statistical-significance machinery is being built.

10.3 Override Workflow

- Every gate carries an override path with required justification.
- Overrides are logged with actor, reason, and time of expiration.
- Gates that cannot be bypassed get disabled by teams under deadline pressure; the framework treats override as a first-class feature with audit and accountability rather than denying it.

10.4 CI Integration

- GitHub App, GitLab integration, and generic webhook surfaces.
- Gates appear as named status checks on the PR; failure writes a structured comment naming the failing metric, phase, and (for load gates) the VU count at which the failure occurred.
- A CLI tool allows local pre-merge gate checks against staging.

11. Operational Concerns

11.1 Reliability and Reproducibility

Performance testing is notoriously flaky; the framework is engineered for it from day one.

- Run each script N times in stability mode; report median and p75; discard cold-cache outliers when configured.
- Pin engine versions (Chromium, Lighthouse, k6) per project so upgrades do not silently change results. Engine version is a baseline-invalidation trigger (§11.4).
- Network emulation via consistent profiles (4G, 3G, cable) using throttling at the proxy or container network namespace, not just CDP.
- Workers run in identical container images, scheduled with anti-affinity, on dedicated nodes (§4.3) to reduce noisy-neighbor effects.
- Track and report run variance — if variance is high, surface it explicitly rather than pretending a single number is truth.
- Load runs add their own reliability concerns; see §12.6.

11.2 Security and Multi-Tenancy

- SSO/OIDC, RBAC at project level (viewer, editor, admin, gate-bypasser).
- Secrets management for test credentials (Vault or cloud secret manager); credentials are referenced by name in scripts, never embedded.
- Artifact encryption at rest; signed URLs for downloads.
- Audit log for gate overrides, budget changes, script edits, and policy-envelope decisions.
- Data residency: per-region storage support for testing customer-facing sites in regulated geographies.

11.3 Compliance and Privacy

- LLM-off mode is the primary privacy control: projects can disable NL authoring entirely, removing any path by which target page content reaches an external model.
- Self-hosted model option for projects that need NL authoring under privacy constraints.
- PII redaction in HARs and traces — request and response bodies are not stored by default; opt-in body capture is project-controlled and goes through a redaction filter.
- Retention controls per data type (per §6.7); shortest retention applies to artifacts containing potential PII.

11.4 Baseline Management

Baselines are central to baseline-relative gates and anomaly detection; their lifecycle deserves explicit treatment.

Lifecycle

- A baseline is computed from a sample of recent stable runs in the relevant scope (project / script / metric) and seasonality bucket.

- Stable means: not invalidated by deploy / rollback / incident / engine version change, and not flagged by the analytics service as anomalous.
- A new baseline is formed when the stable sample for a bucket reaches the configured target size (typically 30–50 runs), not after a fixed time window. This is sample-based stabilization.
- Once formed, the baseline is the active baseline for that scope and bucket until an invalidation trigger fires.

Invalidation Triggers

- Deploy — a deploy event on the target environment invalidates baselines for that environment; the system enters a stabilization phase until the next sample-based baseline forms.
- Rollback — a rollback event invalidates baselines and additionally restores the prior baseline as a candidate; if the rollback is confirmed stable, the restored baseline becomes active.
- Incident — an open incident pauses baseline updates for the affected environment until the incident closes; runs during the incident are not contributed to baseline samples.
- Manual reset — admins can invalidate a baseline explicitly with logged justification.
- Engine version change — a Chromium / Lighthouse / k6 version bump invalidates baselines automatically since measurements may shift across engine versions.

Per-Environment Baselines

- Staging and production baselines are tracked separately and never mixed.
- A regression in staging does not invalidate the production baseline and vice versa.
- Cross-environment comparisons are explicit in the dashboard; the framework does not implicitly trust staging baselines as production proxies.

Seasonality

- Seasonality is stored as schema fields on the baseline row: day_of_week, hour_of_day (binned), and an optional custom bucket (e.g., before / during / after a marketing campaign).
- Each (scope, bucket) combination is a separate baseline row; a 24/7 critical-path comparison uses the matching bucket for the current run.

Baseline Confidence

- Every baseline carries a confidence score derived from sample size and observed variance.
- Low-confidence baselines warn in the dashboard and softfail baseline-relative gates (warn-only) until enough samples accumulate to raise confidence above the threshold.

11.5 Cost and Quota Management

Per-Run Cost Estimation

- Every run carries a cost estimate computed pre-dispatch: engine type, expected duration, VU count (for load runs), and required artifact retention.

- The estimate is shown in the manual run launcher and on the PR check; the run is not dispatched if it would exceed the project's remaining quota.
- Actual cost is computed post-run and reconciled against the estimate; persistent estimation drift triggers a recalibration alert for the operator.

Quotas

- Per-project quotas for: total runs per day, total VU-minutes per day (load runs), total LLM tokens per day (NL authoring), and total artifact storage.
- Override workflow: project owners can request quota increases with a justification that goes to platform admins; emergency overrides are time-limited and audited.
- New projects start with conservative defaults sized for typical small-team usage; quotas grow with adoption.

Cost Drivers

Driver	Typical Share	Mitigation
Worker compute (Playwright + Lighthouse)	30–40%	Spot/preemptible nodes, aggressive autoscaling down
Worker compute (k6 browser load runs)	20–30%	Dedicated load-run quotas; off-hours scheduling for large profiles
ClickHouse storage	10–20%	Cardinality controls (§6.6), tiered retention (§6.7)
LLM tokens (NL authoring)	5–15%	Per-project rate limits, LLM-off mode, cached generations
WPT private agents	5–10%	Opt-in usage, rate limits, scheduled deep dives only
Object store (traces, HARs, videos)	5–10%	Lifecycle policies, opt-in body capture

11.6 Storage Retention Tiers

Consolidated retention policy across all stores:

Tier	Backend	Default Retention	Significant-Run Retention
Hot	ClickHouse SSD	14 days	90 days
Warm	ClickHouse HDD / compressed	180 days	180 days
Cold	S3 Parquet	2 years	2 years
Artifacts (HAR/trace/video)	S3 with lifecycle policy	30 days	180 days
Prometheus (live load-run data)	TSDB	14 days raw, 90 days downsampled	Same

12. Concurrent Load Testing and UI-Server Resource Correlation

Single-session synthetic measurement tells you how fast the frontend renders for one user with no contention. It does not tell you whether the page still renders fast when ten thousand users hit it simultaneously, or whether the SSR fleet is one autoscaling step away from saturation. Version 1.3 adds a load-mode execution path that drives concurrent real-browser sessions, ingests UI-tier server resource metrics in the same time window, and correlates them on a single dashboard view.

> **Why This Matters**
>
> Frontend performance under concurrent load is not a backend load-test concern; it is a frontend concern with a server-side dependency. A page that holds LCP under 2.5 s for one user can blow past 5 s at 100 VUs because SSR queues form, event-loop lag spikes, and TTFB grows even though the rendering pipeline on the client is unchanged. Without load measurement and server correlation, this regression is invisible until production traffic finds it.

12.1 Scope

This section adds the following to the framework:

- k6 browser as a fourth engine (see §3) for concurrent-user load testing with real Chromium.
- A load-profile execution mode alongside the existing stability / deep / scheduled modes.
- UI-tier server telemetry ingestion — node_exporter (OS), prom-client (Node.js), and nginx-prometheus-exporter (static delivery).
- A load-correlation dashboard view stacking frontend timings and server resources on a shared time axis.
- Load-aware quality gates (VU-parameterized thresholds).
- Load-aware ML attribution that distinguishes capacity-driven regressions from code-driven ones.
- Load-specific reliability handling — load-generator monitoring, cache-state control, cold-start coverage.

This is a deliberate scope boundary: the framework loads UI-tier servers because they are inseparable from frontend performance. Deeper load testing of business-logic services and data tiers belongs to existing backend load-test tooling.

12.2 Load Profiles

A load profile is a declarative specification of how virtual users (VUs) ramp over time. Profiles are first-class objects in the framework, stored alongside scripts and versioned with them. The orchestrator dispatches a load run by combining a script, a profile, an environment, and a list of UI-server targets.

Profile Types

Profile	Shape	Use Case
Smoke	1 VU, 1 iteration	Sanity check that the load script runs at all in the target environment
Baseline-load	Ramp to expected production peak, hold for 10 min, ramp down	Verify the system handles expected traffic
Stress	Ramp past expected peak until SLO breach, then back down	Find the knee in the performance curve
Spike	Instant jump from low to high VUs, hold briefly	Tests autoscaling responsiveness and cold-start behavior
Soak	Sustained moderate load for several hours	Surfaces memory leaks, cache eviction effects, slow degradation
Step	Increment VUs in fixed steps with hold periods between	Resolves the load level at which each metric degrades

Profile Specification

```
{
  "id": "checkout_baseline_load",
  "script_id": "checkout_flow",
  "engine": "k6_browser",
  "stages": [
    { "duration": "2m", "target_vus": 20 },
    { "duration": "5m", "target_vus": 50 },
    { "duration": "10m", "target_vus": 100 },
    { "duration": "3m", "target_vus": 100 },
    { "duration": "2m", "target_vus": 0 }
  ],
  "ui_server_targets": [
    "ssr-prod-{1..6}.internal:9100",
    "ssr-prod-{1..6}.internal:9229"
  ],
  "cache_state": "warm",
  "thresholds": {
    "browser_web_vital_lcp": ["p(95)<2500 at vus<=80", "p(95)<3500 at vus<=120"],
    "browser_web_vital_inp": ["p(95)<200"]
  }
}
```

The same canonical script (from §7.4) is referenced; the profile adds VU staging, the target server list for telemetry scraping, and load-aware thresholds. This keeps script authoring unified: one canonical script serves both single-session stability runs and concurrent load runs.

12.3 UI-Server Telemetry

Frontend metrics tell you what happened on the client. Server metrics tell you why. The framework scrapes three classes of metrics from every UI-tier server during a load run:

OS-Level (node_exporter)

- CPU usage broken out by mode (user, system, iowait, steal) per core.
- Memory: RSS, available, buffers, cache, swap usage and rate.

- Disk I/O: throughput, IOPS, latency, queue depth.
- Network: throughput, packet rate, retransmits, connection counts.
- File descriptors and socket counts — common saturation points for high-concurrency Node servers.

Application-Level (prom-client for Node.js)

- Event-loop lag — the single most diagnostic Node.js metric under load.
- Heap size, GC pause time and frequency, old-generation vs new-generation pressure.
- Active handles and active requests (libuv counters).
- HTTP request duration histograms, broken out by route.
- Custom application timings: SSR render duration by route, ISR cache hit/miss, middleware duration, downstream service latency.

Static Delivery (nginx-prometheus-exporter)

- Request rate, response status distribution.
- Connection counts (active, reading, writing, waiting).
- Upstream latency and error rate (when nginx is fronting an SSR app).
- Cache hit ratio for nginx proxy_cache configurations.

Load-Generator Self-Monitoring

The load-generator hosts themselves run node_exporter; their CPU, memory, and network usage are scraped on the same schedule as the target servers. This is required to detect generator saturation, which would otherwise contaminate frontend timing measurements (slow page loads attributed to the target may in fact be the generator running out of headroom). A run is flagged with a warning if any generator exceeds 80% CPU or 90% memory.

12.4 Telemetry Collection Pipeline

During a load run:

5. The orchestrator validates that all target endpoints (UI servers and load generators) are reachable and have a recent successful Prometheus scrape; if any are missing the run aborts with a configuration error rather than producing partial data.
6. The orchestrator records the load-run window's start_time and target_list to ClickHouse before dispatching workers.
7. Prometheus scrapes all targets at a fixed 5-second interval (configurable, default chosen to balance fidelity and load on the target servers themselves).
8. k6 browser workers run the load profile; k6 pushes frontend metrics to the same Prometheus via remote write, time-aligned with server scrapes.
9. On run completion, a Prometheus-to-ClickHouse exporter copies the load-run window's data (filtered to the recorded target list) into the server_resources ClickHouse table.
10. Artifacts (k6 raw output, server-metric snapshot in Parquet) are uploaded to object storage.

Time alignment is verified at run start: the orchestrator queries every target's local clock via a small endpoint or via Prometheus's scrape-time, computes skew against the orchestrator's

NTP-synced clock, and aborts the run if any host exceeds 100 ms drift. This guards against the most common source of bogus correlations.

12.5 Load-Correlation Dashboard View

A new dashboard view, accessible from any load run in the runs list, presents stacked time-series panels on a single shared time axis:

11. Test context — VU count over time, requests per second, errors per second. Establishes the "what was happening" baseline.
12. Frontend timings — p50, p95, p99 of LCP, INP, FCP, TTFB.
13. Frontend errors — page-load failures, JS errors caught, navigation timeouts.
14. UI-server response — TTFB from k6 alongside SSR render duration from prom-client; visualizes the split between server-side rendering time and network/transit.
15. UI-server CPU and memory — per host, with the host fleet aggregated for the fleet view.
16. UI-server runtime — Node.js event-loop lag, GC pause time and frequency, heap size.
17. UI-server I/O — network throughput, open connections, file-descriptor counts.
18. Load-generator health — CPU and memory on every load-generator host as a sanity check.

Ramp-stage transitions are annotated on every panel. The dashboard supports brushing (selecting a sub-window) to drill in: a brushed range filters the request waterfall to requests dispatched in that window, the trace links to requests served during that window, and the anomaly attribution rebuilds against the brushed window.

12.6 Load-Specific Reliability

Load runs introduce reliability concerns distinct from single-session synthetic:

Load-Generator Saturation

If the load generator runs out of CPU, page loads will look slow even when the target system is healthy. Mitigations:

- Hard concurrency cap per generator host based on observed Chromium footprint (typically 50–100 VUs per 16-core, 32 GB host; tuned per project).
- Automatic generator pool sizing — the orchestrator dispatches load runs across enough generator hosts to keep projected per-host VU count under the cap.
- Live generator monitoring with run-level warnings if any generator exceeds 80% CPU or 90% memory.
- Pre-run smoke check — a low-VU profile validates that the generator pool is sized correctly before the real ramp begins.

Cache-State Variance

Cold caches produce dramatically different server-side numbers than warm caches; a load run's results are only comparable to another if cache state is controlled.

- Profiles declare cache_state: cold, warm, or production-replay.
- cold — the orchestrator triggers a documented cache-flush hook on the target before the run starts; the run captures the worst-case capacity envelope.

- warm — a low-VU warmup phase runs before the metrics window opens; the run captures steady-state capacity.
- production-replay — for projects that can shadow real traffic, the framework replays a recorded request pattern to warm caches realistically.

Autoscaling Lag

If the target system autoscales, spike profiles will catch a cold-start window during which performance is degraded. This is by design; the dashboard annotates autoscaling events when an integration with the cloud provider's autoscaling API is configured.

Variance Reporting

Load runs report variance per VU level, not just aggregate variance. A flat p95 LCP across a ramp is the desired outcome; a growing p95 with growing VU count is a capacity finding. The dashboard surfaces this distinction explicitly.

12.7 Load-Aware Quality Gates

Load gates evaluate thresholds parameterized by VU count. The framework supports two forms:

Tiered Thresholds

- Different thresholds at different load levels: "p95 LCP under 2.5 s up to 80 VUs, under 3.5 s up to 120 VUs."
- A run that violates a tier at its VU level fails the gate; the failure comment names the VU level and the violated tier.

Capacity Floors

- Minimum sustainable load before a SLO is breached: "the system must sustain at least 100 VUs while keeping p95 LCP under 2.5 s."
- Useful for capacity-planning gates triggered manually or on a scheduled cadence rather than per-PR.

Server-Resource Floors

- Optional gate conditions on UI-server resources: "CPU must not exceed 80% sustained at target load," "event-loop lag p95 under 50 ms at target load."
- These complement frontend thresholds; a deploy that holds LCP under budget by adding workers should still trigger a capacity-floor violation if per-host resources saturated.

Load gates are not on the per-PR critical path by default — load runs are too expensive to run on every PR. They are typically wired into pre-release gates, scheduled nightly gates, or manual capacity-verification gates. Per-PR gating remains single-session synthetic by default.

12.8 Load-Aware ML Attribution

The analytics service treats UI-server resource series as additional features when attributing a load-run regression:

- If LCP degrades smoothly with VU count and SSR render duration tracks it, attribution names server-side rendering as the bottleneck and indicates a capacity finding.

- If LCP cliffs at a specific VU count and CPU pegs at that moment, attribution names CPU saturation.
- If TTFB grows but client-side timings are stable, attribution names server response, not client rendering.
- If event-loop lag spikes while CPU looks healthy, attribution names synchronous blocking or GC pressure rather than raw capacity.
- If LCP grows but no server resource correlates, attribution names a client-side cause — often payload growth that increased parsing time even though server-side throughput was unchanged.

These attributions are heuristics, not certainties. The explanation surfaces the evidence ("event-loop lag rose from 5 ms to 45 ms at VU=80, coincident with the LCP cliff") so the user can verify or refute.

12.9 Worked Example

A team is preparing to release a checkout redesign. Their pre-release gate includes a baseline-load profile against the staging UI-tier (six SSR pods, each 4 vCPU / 8 GB).

19. The orchestrator dispatches three load-generator hosts, each capped at 50 VUs, for a total target of 100 concurrent VUs.
20. Smoke profile passes; ramp begins. At VU=40, p95 LCP is 1.9 s — within budget. SSR render duration is 220 ms, event-loop lag is 8 ms.
21. At VU=80, p95 LCP rises to 2.4 s. SSR render duration is 380 ms; event-loop lag spikes to 65 ms; CPU on three of six pods crosses 85%. The dashboard annotates the moment with a warning.
22. At VU=100, p95 LCP crosses 3.1 s, violating the tier-2 threshold (3.5 s ceiling at 120 VUs was set; tier-1 was 2.5 s at 80 VUs, also violated for the 80–100 range).
23. The gate fails with the comment: "p95 LCP=3.1s at VUs=100 exceeds tier (2.5s up to 80 VUs). Attribution: SSR render duration (380 ms, +73%) and event-loop lag spike (65 ms p95) on ssr-staging-{2,4,5}. CPU on those three pods sustained >85%. Suspected cause: server-side capacity, not client-side rendering. Recommend: profile SSR render path for the redesigned cart component, or scale staging fleet to mirror production replica count."
24. The team opens the load-correlation view, brushes the VU=80 → VU=100 window, sees that the heaviest SSR route is the new cart sidebar, and traces a backend distributed trace from one slow request to a synchronous database call that previously returned from cache.
25. The team fixes the cache miss, re-runs the load profile, and the gate passes. The release proceeds.

This is what the framework adds in v1.3: not just "this PR slowed the page down," but "this PR slowed the page down at 80 concurrent users because of an SSR-side cache miss, and here is the trace that proves it."

13. Tech Stack

The recommended stack favors mature, widely-deployed open-source components. Specific choices are not load-bearing — the architecture supports substitutions — but each pick reflects an operational trade-off worth naming.

13.1 Selected Components

Layer	Choice	Rationale
Frontend	Next.js + TypeScript, TanStack Query, Tailwind, Recharts/visx, Monaco editor	Modern stack, strong TypeScript story, rich charting for time-series
Backend services	Go or Node.js (NestJS)	Go preferred for orchestrator and telemetry ingestion (throughput); Node.js acceptable for API surface
ML service	Python	Ecosystem alignment for change-point and attribution work
Orchestration	Kubernetes + KEDA/HPA	Per-engine node pools, anti-affinity, autoscaling
Workflow engine	Argo Workflows or Temporal	Long-running multi-stage runs benefit from durable workflow execution
Queue	Kafka (preferred) or Redis Streams	Kafka for production scale; Redis Streams acceptable for smaller deployments
Metadata DB	Postgres	Standard relational store
Metrics + per-request data	ClickHouse	Designed for high-volume time-series with fast aggregations
Live metrics	Prometheus	Native target for k6, node_exporter, prom-client, nginx-exporter
Object store	S3-compatible	Artifacts, traces, HARs, videos
Observability	OpenTelemetry + Grafana + Loki	Internal ops dashboards and logs
CI integrations	GitHub App + GitLab integration + generic webhook + CLI	Covers majority of pipelines

13.2 Engine Runtime Components

- Playwright workers — containerized with pinned Chromium version; deployed to a dedicated synthetic node pool.
- Lighthouse — invoked from inside Playwright workers; produces JSON reports stored in object storage.
- k6 browser — containerized with pinned Chromium; deployed to a dedicated load-generator node pool sized differently from synthetic workers.
- WPT private agents — managed as a small, separate fleet of long-lived agents with their own browser/network profile management.

- sitespeed.io — scheduled runs from a small fleet of continuous-coverage workers.
- node_exporter, prom-client, nginx-prometheus-exporter — installed on every UI server under test as part of the project's environment configuration.

14. Phased Delivery Plan

A pragmatic roadmap from MVP through GA, with the v1.3 load-testing capabilities introduced in Phase 3.5 once the single-session framework is operationally stable.

14.1 Phase 0 — Foundations (Weeks 1–3)

- Architecture spike.
- Engine PoC: Playwright + Lighthouse runs in a container, emitting metrics.
- Data-model draft.
- Repo scaffolding and CI setup.

14.2 Phase 1 — MVP (Weeks 4–10)

Goal: usable end-to-end by an internal pilot team.

- Single engine: Playwright + Lighthouse, stability mode only.
- Basic dashboard: list runs, view metrics, simple trend chart.
- Postgres + object store; no ClickHouse yet.
- Manual and scheduled runs; GitHub check integration.
- Hard-threshold quality gates only with 3-of-5 quorum to control flakiness.

Out of Phase 1

- No WPT integration.
- No self-healing scripts.
- No ML attribution.
- No statistical or baseline-relative gates.
- No Server-Timing or APM integration.
- No multi-region execution.
- No load mode (deferred to Phase 3.5).

14.3 Phase 2 — Depth (Weeks 11–18)

- Add WPT private-instance integration (opt-in, rate-limited).
- Add Compare view and Waterfall view with filmstrip.
- Add ClickHouse for metrics and per-request data with cardinality controls.
- Multi-engine TestRunner abstraction (Playwright, Lighthouse, WPT, sitespeed.io).
- Add RBAC, script versioning via Git, Slack notifications.
- Add baseline-relative gates with sample-based baseline stabilization.

14.4 Phase 3 — Intelligence (Weeks 19–26)

- Statistical baselines with per-environment, per-seasonality bucket storage.
- Change-point detection (CUSUM, EWMA).

- Anomaly feed and dashboard view.
- Heuristic root-cause attribution (phase-level and request-level).
- Executive reports and per-team scorecards.
- Scheduled digests via Slack and email.
- Natural-language test authoring with the six-stage pipeline, runtime automation policy envelope, and semantic post-conditions for self-healing.

14.5 Phase 3.5 — Concurrent Load Testing (Weeks 27–34) — added in v1.3

Goal: deliver the v1.3 capabilities so frontend performance can be measured under realistic load and correlated to UI-server resources.

- Integrate k6 browser as the load engine; add it to the TestRunner abstraction.
- Implement the load-profile object: stages, target_vus, cache_state, ui_server_targets.
- Stand up the dedicated load-generator node pool with anti-affinity and per-host concurrency caps.
- UI-server telemetry: configure Prometheus scraping for node_exporter, prom-client, and nginx-prometheus-exporter; build the Prometheus-to-ClickHouse exporter for load-run snapshots.
- server_resources table in ClickHouse; load_runs metadata in Postgres.
- Load-correlation dashboard view with stacked time-series, ramp-stage annotations, and brushing.
- Load-generator self-monitoring with run-level saturation warnings.
- Cache-state controls (cold / warm / production-replay) and pre-run validation.
- Engine-specific concurrency caps in the orchestrator so load runs do not displace per-PR stability runs.
- Cost estimation extended to cover VU-minutes; quota enforcement for load runs.
- Tiered load-aware gate type (VU-parameterized thresholds).

14.6 Phase 4 — GA, Scale, and Advanced Intelligence (Weeks 35+)

- SHAP-based feature attribution once enough historical data exists.
- Prophet-style forecasting on trend charts.
- Multi-geo WPT agents.
- Multi-region load generation for k6 browser.
- Load-aware ML attribution incorporating server-resource series.
- RUM and CrUX ingestion.
- Public API and SDK maturity.
- Audit and compliance hardening.
- Server-resource floor gates (CPU and event-loop lag conditions on load gates).
- Capacity-floor gates and capacity-planning reports.

15. Success Metrics

Measure the framework itself, not just the systems it observes.

Metric	Target
Test author time-to-first-run	Under 10 minutes
Mean time to detect regression after deploy	Under 15 minutes
False-positive rate on gates	Under 5% — critical for developer trust
Adoption — % of production-deploying services with at least one gated journey	60%+ by end of year following GA
Outcomes — trend of p75 Core Web Vitals across covered surfaces	Improving year-over-year
Time-to-root-cause on regression (gate-fail to phase-named comment)	Under 5 minutes where data exists
Attribution accuracy — ML root-cause agrees with human RCA	80%+ agreement on sampled incidents
Load-run reliability — % of load runs with no generator saturation warning (v1.3)	95%+
Load-attribution accuracy — server-side vs client-side calls match human review (v1.3)	85%+ on sampled load-run incidents

16. Risks and Mitigations

Risk	Mitigation
Noise and flakiness eroding trust	N-run medians, 3-of-5 quorum (Phase 1), statistical gates (Phase 3), explicit variance reporting
Engine sprawl complicating maintenance	Strong TestRunner abstraction and a blessed default (Playwright + Lighthouse) for most cases
ML producing unexplainable alerts	Always ship human-readable explanations; user feedback loop on attribution accuracy
WPT operational cost	Use WPT selectively (scheduled deep dives, key journeys), not on every PR
Adoption friction	Templates, codegen recorder, copy-paste CI snippets; invest in onboarding UX early
LLM cost or privacy concerns	LLM-off mode as primary control; self-hosted model option; per-project rate limits
NL-generated scripts causing semantic drift	Mandatory semantic post-conditions on every step; Drifted vs Healed states distinct; cohort approval for bulk healing
ClickHouse cardinality explosion	Top-N persistence, third-party aggregation, sampling for continuous synthetic, significance-aware hot-tier retention
Baselines never stabilize in high-velocity environments	Sample-based stabilization rather than fixed-time windows; per-bucket baselines for seasonality
Orchestrator starvation under thundering herd	Per-project concurrency caps, weighted-fair queue, engine-specific quotas, LLM rate limits
Load runs displacing per-PR runs (v1.3)	Engine-specific concurrency caps; load-run quotas separate from stability-run quotas
Load-generator saturation contaminating measurements (v1.3)	Per-host VU caps, automatic generator pool sizing, live generator monitoring, run-level saturation warnings
Cache-state variance making load runs non-comparable (v1.3)	Profiles declare cache_state explicitly; cold/warm/production-replay options with pre-run validation
Cost growth on load runs (v1.3)	VU-minute quotas; off-hours scheduling for large profiles; load runs not on the per-PR critical path by default